

Numerical Recipes Software 2007, “Code Implementation for Fitexy,” *Numerical Recipes Web-note No. 19*, at <http://www.nr.com/webnotes?19> [11]

15.4 General Linear Least Squares

An immediate generalization of §15.2 is to fit a set of data points (x_i, y_i) to a model that is not just a linear combination of 1 and x (namely $a + bx$), but rather a linear combination of *any* M specified functions of x . For example, the functions could be $1, x, x^2, \dots, x^{M-1}$, in which case their general linear combination,

$$y(x) = a_0 + a_1x + a_2x^2 + \dots + a_{M-1}x^{M-1} \quad (15.4.1)$$

is a polynomial of degree $M - 1$. Or, the functions could be sines and cosines, in which case their general linear combination is a Fourier series. The general form of this kind of model is

$$y(x) = \sum_{k=0}^{M-1} a_k X_k(x) \quad (15.4.2)$$

where the quantities $X_0(x), \dots, X_{M-1}(x)$ are arbitrary fixed functions of x , called the *basis functions*.

Note that the functions $X_k(x)$ can be wildly nonlinear functions of x . In this discussion, “linear” refers only to the model’s dependence on its *parameters* a_k .

For these linear models we generalize the discussion of the previous section by defining a merit function

$$\chi^2 = \sum_{i=0}^{N-1} \left[\frac{y_i - \sum_{k=0}^{M-1} a_k X_k(x_i)}{\sigma_i} \right]^2 \quad (15.4.3)$$

As before, σ_i is the measurement error (standard deviation) of the i th data point, presumed to be known. If the measurement errors are not known, they may all (as discussed at the end of §15.1) be set to the constant value $\sigma = 1$.

Once again, we will pick as best parameters those that minimize χ^2 . There are several different techniques available for finding this minimum. Two are particularly useful, and we will discuss both in this section. To introduce them and elucidate their relationship, we need some notation.

Let $\mathbf{A}$ be a matrix whose $N \times M$ components are constructed from the M basis functions evaluated at the N abscissas x_i , and from the N measurement errors σ_i , by the prescription

$$A_{ij} = \frac{X_j(x_i)}{\sigma_i} \quad (15.4.4)$$

The matrix $\mathbf{A}$ is called the *design matrix* of the fitting problem. Notice that in general $\mathbf{A}$ has more rows than columns, $N \geq M$, since there must be more data points than model parameters to be solved for. (You can fit a straight line to two points, but not a very meaningful quintic!) The design matrix is shown schematically in Figure 15.4.1.

$$\begin{array}{c}
 \longleftarrow \text{basis functions} \longrightarrow \\
 X_0(\) \quad X_1(\) \quad \cdots \quad X_{M-1}(\) \\
 \\
 \begin{array}{c}
 \uparrow \\
 \text{data points} \\
 \downarrow
 \end{array}
 \begin{pmatrix}
 x_0 & \frac{X_0(x_0)}{\sigma_0} & \frac{X_1(x_0)}{\sigma_0} & \cdots & \frac{X_{M-1}(x_0)}{\sigma_0} \\
 x_1 & \frac{X_0(x_1)}{\sigma_1} & \frac{X_1(x_1)}{\sigma_1} & \cdots & \frac{X_{M-1}(x_1)}{\sigma_1} \\
 \vdots & \vdots & \vdots & \ddots & \vdots \\
 x_{N-1} & \frac{X_0(x_{N-1})}{\sigma_{N-1}} & \frac{X_1(x_{N-1})}{\sigma_{N-1}} & \cdots & \frac{X_{M-1}(x_{N-1})}{\sigma_{N-1}}
 \end{pmatrix}
 \end{array}$$

Figure 15.4.1. Design matrix $\mathbf{A}$ for the least-squares fit of a linear combination of M basis functions to N data points. The matrix elements involve the basis functions evaluated at the values of the independent variable at which measurements are made and the standard deviations of the measured dependent variable. The measured values of the dependent variable do not enter the design matrix.

Also define a vector $\mathbf{b}$ of length N by

$$b_i = \frac{y_i}{\sigma_i} \quad (15.4.5)$$

and denote the M vector whose components are the parameters to be fitted, $a_0, \dots, a_{M-1}$, by $\mathbf{a}$.

15.4.1 Solution by Use of the Normal Equations

The minimum of (15.4.3) occurs where the derivative of χ^2 with respect to all M parameters a_k vanishes. Specializing equation (15.1.8) to the case of the model (15.4.2), this condition yields the M equations

$$0 = \sum_{i=0}^{N-1} \frac{1}{\sigma_i^2} \left[y_i - \sum_{j=0}^{M-1} a_j X_j(x_i) \right] X_k(x_i) \quad k = 0, \dots, M-1 \quad (15.4.6)$$

Interchanging the order of summations, we can write (15.4.6) as the matrix equation

$$\sum_{j=0}^{M-1} \alpha_{kj} a_j = \beta_k \quad (15.4.7)$$

where

$$\alpha_{kj} = \sum_{i=0}^{N-1} \frac{X_j(x_i)X_k(x_i)}{\sigma_i^2} \quad \text{or, equivalently,} \quad \boldsymbol{\alpha} = \mathbf{A}^T \cdot \mathbf{A} \quad (15.4.8)$$

an $M \times M$ matrix, and

$$\beta_k = \sum_{i=0}^{N-1} \frac{y_i X_k(x_i)}{\sigma_i^2} \quad \text{or, equivalently,} \quad \boldsymbol{\beta} = \mathbf{A}^T \cdot \mathbf{b} \quad (15.4.9)$$

a vector of length M .

The equations (15.4.6) or (15.4.7) are called the *normal equations* of the least-squares problem. They can be solved for the vector of parameters $\mathbf{a}$ by the standard methods of Chapter 2, notably *LU* decomposition and backsubstitution, Choleksy decomposition, or Gauss-Jordan elimination. In matrix form, the normal equations can be written as either

$$\boldsymbol{\alpha} \cdot \mathbf{a} = \boldsymbol{\beta} \quad \text{or as} \quad (\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{a} = \mathbf{A}^T \cdot \mathbf{b} \quad (15.4.10)$$

The inverse matrix $\mathbf{C} \equiv \boldsymbol{\alpha}^{-1}$, called the covariance matrix, is closely related to the probable (or, more precisely, *standard*) uncertainties of the estimated parameters $\mathbf{a}$. To estimate these uncertainties, consider that

$$a_j = \sum_{k=0}^{M-1} \alpha_{jk}^{-1} \beta_k = \sum_{k=0}^{M-1} C_{jk} \left[\sum_{i=0}^{N-1} \frac{y_i X_k(x_i)}{\sigma_i^2} \right] \quad (15.4.11)$$

and that the variance associated with the estimate a_j can be found as in (15.2.7) from

$$\sigma^2(a_j) = \sum_{i=0}^{N-1} \sigma_i^2 \left(\frac{\partial a_j}{\partial y_i} \right)^2 \quad (15.4.12)$$

Note that α_{jk} is independent of y_i , so that

$$\frac{\partial a_j}{\partial y_i} = \sum_{k=0}^{M-1} C_{jk} X_k(x_i) / \sigma_i^2 \quad (15.4.13)$$

Consequently, we find that

$$\sigma^2(a_j) = \sum_{k=0}^{M-1} \sum_{l=0}^{M-1} C_{jk} C_{jl} \left[\sum_{i=0}^{N-1} \frac{X_k(x_i) X_l(x_i)}{\sigma_i^2} \right] \quad (15.4.14)$$

The final term in brackets is just the matrix $\boldsymbol{\alpha}$. Since this is the matrix inverse of $\mathbf{C}$, (15.4.14) reduces immediately to

$$\sigma^2(a_j) = C_{jj} \quad (15.4.15)$$

In other words, the diagonal elements of $\mathbf{C}$ are the variances (squared uncertainties) of the fitted parameters $\mathbf{a}$. It should not surprise you to learn that the off-diagonal elements C_{jk} are the covariances between a_j and a_k (cf. 15.2.10); but we shall defer discussion of these to §15.6.

We will now give a routine that implements the above formulas for the general linear least-squares problem, by the method of normal equations. Since we wish to compute not only the solution vector $\mathbf{a}$ but also the covariance matrix $\mathbf{C}$, it is most convenient to use Gauss-Jordan elimination (routine `gaussj` of §2.1) to perform the linear algebra. The operation count in this application is no larger than that for LU decomposition. If you have no need for the covariance matrix, however, you can save a factor of 3 on the linear algebra by switching to LU decomposition, without computation of the matrix inverse. In theory, since $\mathbf{A}^T \cdot \mathbf{A}$ is positive-definite, Cholesky decomposition is the most efficient way to solve the normal equations. However, in practice, most of the computing time is spent in looping over the data to form the equations, and Gauss-Jordan is quite adequate.

We need to warn you that the solution of a least-squares problem directly from the normal equations is rather susceptible to roundoff error, because the condition number of the matrix α is the square of the condition number of $\mathbf{A}$. An alternative, and preferred, technique involves QR decomposition (§2.10, §11.4, and §11.7) of the design matrix $\mathbf{A}$. This is essentially what we did at the end of §15.2 for fitting data to a straight line, but without invoking all the machinery of QR to derive the necessary formulas. Later in this section, we will discuss other difficulties in the least-squares problem, for which the cure is *singular value decomposition* (SVD), of which we give an implementation. It turns out that SVD also fixes the roundoff problem, so it is our recommended technique for all but “easy” least-squares problems. It is for these easy problems that the following routine, which solves the normal equations, is intended.

The object `Fitlin`, below, has one “value-added feature” that can be quite useful in practical work: Frequently it is a matter of art to decide which parameters a_k in a model should be fit from the data set, and which should be held constant at fixed values, for example values predicted by a theory or measured in a previous experiment. One wants, therefore, to have a convenient means for “freezing” and “un-freezing” the parameters a_k . In the following code, the total number of parameters a_k is denoted `ma` (called M above) and is deduced from the size of the vector that is returned by the user-supplied fitting function routine. The `Fitlin` object maintains a boolean array `ia[0..ma-1]`. Components that are `false` indicate that you want the corresponding elements of the parameter vector `a[0..ma-1]` to be held fixed at their input values. Components that are `true` indicate parameters that should be fitted for. On output, any frozen parameters will have their variances, and all their covariances, set to zero in the covariance matrix.

```
struct Fitlin {
```

`fitlin.h`

Object for general linear least-squares fitting by solving the normal equations, also including the ability to hold specified parameters at fixed, specified values. Call constructor to bind data vectors and fitting functions. Then call any combination of `hold`, `free`, and `fit` as often as desired. `fit` sets the output quantities `a`, `covar`, and `chisq`.

```
    Int ndat, ma;
    VecDoub_I &x,&y,&sig;
    VecDoub (*funcs)(const Doub);
    VecBool ia;
```

```
    VecDoub a;
    MatDoub covar;
    Doub chisq;
```

Output values. `a` is the vector of fitted coefficients, `covar` is its covariance matrix, and `chisq` is the value of χ^2 for the fit.

```
Fitlin(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig, VecDoub funks(const Doub))
```

```
: ndat(xx.size()), x(xx), y(yy), sig(ssig), funcs(funcs) {
```

Constructor. Binds references to the data arrays xx, yy, and ssig, and to a user-supplied function funks(x) that returns a VecDoub containing ma basis functions evaluated at $x = x$. Initializes all parameters as free (not held).

```
    ma = funcs(x[0]).size();
    a.resize(ma);
    covar.resize(ma,ma);
    ia.resize(ma);
    for (Int i=0;i<ma;i++) ia[i] = true;
}
```

```
void hold(const Int i, const Doub val) {ia[i]=false; a[i]=val;}
void free(const Int i) {ia[i]=true;}
```

Optional functions for holding a parameter, identified by a value i in the range 0, ..., ma-1, fixed at the value val, or for freeing a parameter that was previously held fixed. hold and free may be called for any number of parameters before calling fit to calculate best-fit values for the remaining (not held) parameters, and the process may be repeated multiple times. Alternatively, you can set the boolean vector ia directly, before calling fit.

```
void fit() {
```

Solve the normal equations for χ^2 minimization to fit for some or all of the coefficients a[0..ma-1] of a function that depends linearly on a, $y = \sum_i a_i \times \text{funks}_i(x)$. Set answer values for a[0..ma-1], $\chi^2 = \text{chisq}$, and the covariance matrix covar[0..ma-1][0..ma-1]. (Parameters held fixed by calls to hold will return zero covariances.)

```
    Int i,j,k,l,m,mfit=0;
    Doub ym,wt,sum,sig2i;
    VecDoub afunc(ma);
    for (j=0;j<ma;j++) if (ia[j]) mfit++;
    if (mfit == 0) throw("lfit: no parameters to be fitted");
    MatDoub temp(mfit,mfit,0.),beta(mfit,1,0.);
    for (i=0;i<ndat;i++) {
        afunc = funcs(x[i]);
        ym=y[i];
        if (mfit < ma) {
            for (j=0;j<ma;j++)
                if (!ia[j]) ym -= a[j]*afunc[j];
        }
        sig2i=1.0/SQR(sig[i]);
        for (j=0,l=0;l<ma;l++) {
            if (ia[l]) {
                wt=afunc[l]*sig2i;
                for (k=0,m=0;m<1;m++)
                    if (ia[m]) temp[j][k++] += wt*afunc[m];
                beta[j++][0] += ym*wt;
            }
        }
    }
    for (j=1;j<mfit;j++) for (k=0;k<j;k++) temp[k][j]=temp[j][k];
    gaussj(temp,beta);
    for (j=0,l=0;l<ma;l++) if (ia[l]) a[l]=beta[j++][0];
    Spread the solution to appropriate positions in a, and evaluate  $\chi^2$  of the fit.
    chisq=0.0;
    for (i=0;i<ndat;i++) {
        afunc = funcs(x[i]);
        sum=0.0;
        for (j=0;j<ma;j++) sum += a[j]*afunc[j];
        chisq += SQR((y[i]-sum)/sig[i]);
    }
    for (j=0;j<mfit;j++) for (k=0;k<mfit;k++) covar[j][k]=temp[j][k];
    for (i=mfit;i<ma;i++)
        for (j=0;j<i+1;j++) covar[i][j]=covar[j][i]=0.0;
    k=mfit-1;
    for (j=ma-1;j>=0;j--) {
        // Rearrange covariance matrix into the correct order.
```

```

        if (ia[j]) {
            for (i=0;i<ma;i++) SWAP(covar[i][k],covar[i][j]);
            for (i=0;i<ma;i++) SWAP(covar[k][i],covar[j][i]);
            k--;
        }
    }
};

```

Typical use of `Fitlin` will look something like this:

```

const Int npts=...
VecDoub xx(npts),yy(npts),ssig(npts);
...
Fitlin myfit(xx,yy,ssig,cubicfit);
myfit.fit();

```

where (in this example) `cubicfit` is a user-supplied function that might look like this:

```

VecDoub cubicfit(const Doub x) {
    VecDoub ans(4);
    ans[0] = 1.;
    for (Int i=1;i<4;i++) ans[i] = x*ans[i-1];
    return ans;
}

```

15.4.2 Solution by Use of Singular Value Decomposition

In some applications, the normal equations are perfectly adequate for linear least-squares problems. However, in many other cases the normal equations are very close to singular. A zero pivot element may be encountered during the solution of the linear equations (e.g., in `gaussj`), in which case you get no solution at all. Or a very small pivot may occur, in which case you typically get fitted parameters a_k with very large magnitudes that are delicately (and unstably) balanced to cancel out almost precisely when the fitted function is evaluated.

Why does this commonly occur? A mathematical reason is that the condition number of the matrix α is the square of the condition number of $\mathbf{A}$. But an additional physical reason is that, more often than experimenters would like to admit, data do not clearly distinguish between two or more of the basis functions provided. If two such functions, or two different combinations of functions, happen to fit the data about equally well — or equally badly — then the matrix α , unable to distinguish between them, neatly folds up its tent and becomes singular. There is a certain mathematical irony in the fact that least-squares problems are *both* overdetermined (number of data points greater than number of parameters) *and* underdetermined (ambiguous combinations of parameters exist); but that is how it frequently is. The ambiguities can be extremely hard to notice a priori in complicated problems.

Enter singular value decomposition (SVD). This would be a good time for you to review the material in §2.6, which we will not repeat here. In the case of an overdetermined system, SVD produces a solution that is the best approximation in the least-squares sense, cf. equation (2.6.10). That is exactly what we want. In the case of an underdetermined system, SVD produces a solution whose values (for us, the a_k 's) are smallest in the least-squares sense, cf. equation (2.6.8). That is also what we want: When some combination of basis functions is irrelevant to the fit, that

combination will be driven down to a small, innocuous, value, rather than pushed up to delicately canceling infinities.

In terms of the design matrix $\mathbf{A}$ (equation 15.4.4) and the vector $\mathbf{b}$ (equation 15.4.5), minimization of χ^2 in (15.4.3) can be written as

$$\text{find } \mathbf{a} \text{ that minimizes } \chi^2 = |\mathbf{A} \cdot \mathbf{a} - \mathbf{b}|^2 \quad (15.4.16)$$

Comparing to equation (2.6.9), we see that this is precisely the problem that routines in the SVD object are designed to solve. The solution, which is given by equation (2.6.12), can be rewritten as follows: If $\mathbf{U}$ and $\mathbf{V}$ enter the SVD decomposition of $\mathbf{A}$ according to equation (2.6.1), as computed by SVD, then let the vectors $\mathbf{U}_{(i)}$ $i = 0, \dots, M-1$ denote the *columns* of $\mathbf{U}$ (each one a vector of length N), and let the vectors $\mathbf{V}_{(i)}$ $i = 0, \dots, M-1$ denote the *columns* of $\mathbf{V}$ (each one a vector of length M). Then the solution (2.6.12) of the least-squares problem (15.4.16) can be written as

$$\mathbf{a} = \sum_{i=0}^{M-1} \left(\frac{\mathbf{U}_{(i)} \cdot \mathbf{b}}{w_i} \right) \mathbf{V}_{(i)} \quad (15.4.17)$$

where the w_i are, as in §2.6, the singular values calculated by SVD.

Equation (15.4.17) says that the fitted parameters $\mathbf{a}$ are linear combinations of the columns of $\mathbf{V}$, with coefficients obtained by forming dot products of the columns of $\mathbf{U}$ with the weighted data vector (15.4.5). Though it is beyond our scope to prove here, it turns out that the standard (loosely, “probable”) errors in the fitted parameters are also linear combinations of the columns of $\mathbf{V}$. In fact, equation (15.4.17) can be written in a form displaying these errors as

$$\mathbf{a} = \left[\sum_{i=0}^{M-1} \left(\frac{\mathbf{U}_{(i)} \cdot \mathbf{b}}{w_i} \right) \mathbf{V}_{(i)} \right] \pm \frac{1}{w_0} \mathbf{V}_{(0)} \pm \dots \pm \frac{1}{w_{M-1}} \mathbf{V}_{(M-1)} \quad (15.4.18)$$

Here each $\pm$ is followed by a standard deviation. The amazing fact is that, decomposed in this fashion, the standard deviations are all mutually independent (uncorrelated). Therefore they can be added together in root-mean-square fashion. What is going on is that the vectors $\mathbf{V}_{(i)}$ are the principal axes of the error ellipsoid of the fitted parameters $\mathbf{a}$ (see §15.6).

It follows that the variance in the estimate of a parameter a_j is given by

$$\sigma^2(a_j) = \sum_{i=0}^{M-1} \frac{1}{w_i^2} [\mathbf{V}_{(i)}]_j^2 = \sum_{i=0}^{M-1} \left(\frac{V_{ji}}{w_i} \right)^2 \quad (15.4.19)$$

whose result should be identical to (15.4.14). As before, you should not be surprised at the formula for the covariances, here given without proof,

$$\text{Cov}(a_j, a_k) = \sum_{i=0}^{M-1} \left(\frac{V_{ji} V_{ki}}{w_i^2} \right) \quad (15.4.20)$$

We introduced this subsection by noting that the normal equations can fail by encountering a zero pivot. We have not yet, however, mentioned how SVD overcomes this problem. The answer is: If any singular value w_i is zero, its reciprocal

in equation (15.4.18) should be set to zero, not infinity. (Compare the discussion preceding equation 2.6.7.) This corresponds to adding to the fitted parameters $\mathbf{a}$ a *zero* multiple, rather than some random large multiple, of any linear combination of basis functions that are degenerate in the fit. It is a good thing to do!

Moreover, if a singular value w_i is nonzero but very small, you should also define *its* reciprocal to be zero, since its apparent value is probably an artifact of roundoff error, not a meaningful number. A plausible answer to the question “how small is small?” is to edit in this fashion all singular values whose ratio to the largest singular value is less than N times the machine precision ϵ . (This is a more conservative recommendation than the default in §2.6, which scales as $N^{1/2}$.)

There is another reason for editing even *additional* singular values, ones large enough that roundoff error is not a question. Singular value decomposition allows you to identify linear combinations of variables that just happen not to contribute much to reducing the χ^2 of your data set. Editing these can sometimes reduce the probable error errors on your coefficients quite significantly, while increasing the minimum χ^2 only negligibly. We will learn more about identifying and treating such cases in §15.6.

Generally speaking, we recommend that you always use SVD techniques instead of using the normal equations. SVD’s only significant disadvantage is that it requires extra storage of order $N \times M$ for the design matrix and its decomposition. Storage is also required for the $M \times M$ matrix $\mathbf{V}$, but this is instead of the same-sized matrix for the coefficients of the normal equations. SVD can be significantly slower than solving the normal equations; however, its great advantage, that it (theoretically) *cannot fail*, more than makes up for the speed disadvantage.

The following object, `Fitsvd`, has an interface almost identical to `Fitlin`, above. An additional optional parameter in the constructor sets the threshold for editing singular values.

```
struct Fitsvd {
```

Object for general linear least-squares fitting using singular value decomposition. Call constructor to bind data vectors and fitting functions. Then call `fit`, which sets the output quantities `a`, `covar`, and `chisq`.

```
    Int ndat, ma;
```

```
    Doub tol;
```

```
    VecDoub_I *x,&y,&sig;
```

(Why is `x` a pointer? Explained in §15.4.4.)

```
    VecDoub (*funcs)(const Doub);
```

```
    VecDoub a;
```

Output values. `a` is the vector of fitted coefficients,

```
    MatDoub covar;
```

`covar` is its covariance matrix, and `chisq` is the

```
    Doub chisq;
```

value of χ^2 for the fit.

```
    Fitsvd(VecDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig,
```

```
    VecDoub funks(const Doub), const Doub TOL=1.e-12)
```

```
    : ndat(yy.size()), x(&xx), xmd(NULL), y(yy), sig(ssig),
```

```
    funks(funks), tol(TOL) {}
```

Constructor. Binds references to the data arrays `xx`, `yy`, and `ssig`, and to a user-supplied function `funks(x)` that returns a `VecDoub` containing `ma` basis functions evaluated at $x = x$. If `TOL` is positive, it is the threshold (relative to the largest singular value) for discarding small singular values. If it is ≤ 0 , the default value in SVD is used.

```
void fit() {
```

Solve by singular value decomposition the χ^2 minimization that fits for the coefficients `a[0..ma-1]` of a function that depends linearly on `a`, $y = \sum_i a_i \times \text{funks}_i(x)$. Set answer values for `a[0..ma-1]`, `chisq` = χ^2 , and the covariance matrix `covar[0..ma-1][0..ma-1]`.

`fitsvd.h`

```

Int i,j,k;
Doub tmp,thresh,sum;
if (x) ma = funcns((*x)[0]).size();
else ma = funcsmid(row(*xmd,0)).size();           (Discussed in §15.4.4.)
a.resize(ma);
covar.resize(ma,ma);
MatDoub aa(ndat,ma);
VecDoub b(ndat),afunc(ma);
for (i=0;i<ndat;i++) {                           Accumulate coefficients of the
    if (x) afunc=funcns((*x)[i]);                  design matrix.
    else afunc=funcsmid(row(*xmd,i));              (Discussed in §15.4.4.)
    tmp=1.0/sig[i];
    for (j=0;j<ma;j++) aa[i][j]=afunc[j]*tmp;
    b[i]=y[i]*tmp;
}
SVD svd(aa);                                     Singular value decomposition.
thresh = (tol > 0. ? tol*svd.w[0] : -1.);
svd.solve(b,a,thresh);                           Solve for the coefficients.
chisq=0.0;                                         Evaluate chi-square.
for (i=0;i<ndat;i++) {
    sum=0.;
    for (j=0;j<ma;j++) sum += aa[i][j]*a[j];
    chisq += SQR(sum-b[i]);
}
for (i=0;i<ma;i++) {                             Sum contributions to covariance
    for (j=0;j<i+1;j++) {                         matrix (15.4.20).
        sum=0.0;
        for (k=0;k<ma;k++) if (svd.w[k] > svd.tsh)
            sum += svd.v[i][k]*svd.v[j][k]/SQR(svd.w[k]);
        covar[j][i]=covar[i][j]=sum;
    }
}
}
}

```

From here on, code for multidimensional fits, to be discussed in §15.4.4.

```

MatDoub_I *xmd;
VecDoub (*funcsmid)(VecDoub_I &);

Fitsvd(MatDoub_I &xx, VecDoub_I &yy, VecDoub_I &ssig,
VecDoub funks(VecDoub_I &), const Doub TOL=1.e-12)
: ndat(yy.size()), x(NULL), xmd(&xx), y(yy), sig(ssig),
funcsmid(funcns), tol(TOL) {}

```

Constructor for multidimensional fits. Exactly the same as the previous constructor, except that *xx* is now a matrix whose rows are the multidimensional data points and *funks* is now a function of a multidimensional data point (as a *VecDoub*).

```

VecDoub row(MatDoub_I &a, const Int i) {
Utility. Returns the row of a MatDoub as a VecDoub.
    Int j,n=a.ncols();
    VecDoub ans(n);
    for (j=0;j<n;j++) ans[j] = a[i][j];
    return ans;
}
};

```

For degenerate or nearly degenerate problems, if you want to try different singular value thresholds, you call the *Fitsvd* constructor once. Then, as many times as you want, “reach in” and increase *tol*, then call *fit* again and examine the resulting value of *chisq* (and optionally also the covariance matrix). Keep going as long as *chisq* does not increase by too much. To learn what is “too much,” see §15.6; but a few $\times 0.1$ is almost always OK.

15.4.3 Examples

Be aware that some apparently nonlinear problems can be expressed so that they are linear. For example, an exponential model with two parameters a and b ,

$$y(x) = a \exp(-bx) \quad (15.4.21)$$

can be rewritten as

$$\log[y(x)] = c - bx \quad (15.4.22)$$

which is linear in its parameters c and b . (Of course you must be aware that such transformations do not exactly take Gaussian errors into Gaussian errors.)

Also watch out for “nonparameters,” as in

$$y(x) = a \exp(-bx + d) \quad (15.4.23)$$

Here the parameters a and d are, in fact, indistinguishable. This is a good example of where the normal equations will be exactly singular, and where SVD will find a zero singular value. SVD will then make a least-squares choice for setting a balance between a and d (or, rather, their equivalents in the linear model derived by taking the logarithms). However — and this is true whenever SVD gives back a zero singular value — you are better advised to figure out analytically where the degeneracy is among your basis functions, and then make appropriate deletions in the basis set.

We already gave an example of a user-supplied fitting-function routine, `cubicfit`, above. Here are two further examples. First, we trivially generalize `cubicfit` for polynomials of an arbitrary, preset, degree:

```
Int fpoly_np = 10;                                Global variable for the degree plus one. fit\_examples.h

VecDoub fpoly(const Doub x) {
  Fitting routine for a polynomial of degree fpoly_np-1.
  Int j;
  VecDoub p(fpoly_np);
  p[0]=1.0;
  for (j=1;j<fpoly_np;j++) p[j]=p[j-1]*x;
  return p;
}
```

The second example is slightly less trivial. It is used to fit Legendre polynomials up to some order `fleg_n1` to a data set. (Note that, for most uses, the data should satisfy $-1 \leq x \leq 1$.)

```
Int fleg_n1 = 10;                                Global variable for the degree plus one. fit\_examples.h

VecDoub fleg(const Doub x) {
  Fitting routine for an expansion with n1 Legendre polynomials, evaluated using the recurrence
  relation as in §5.4.
  Int j;
  Doub twox,f2,f1,d;
  VecDoub pl(fleg_n1);
  pl[0]=1.;
  pl[1]=x;
  if (fleg_n1 > 2) {
    twox=2.*x;
    f2=x;
    d=1.;
    for (j=2;j<fleg_n1;j++) {
```

```

        f1=d++;
        f2+=twox;
        pl[j]=(f2*pl[j-1]-f1*pl[j-2])/d;
    }
}
return pl;
}

```

15.4.4 Multidimensional Fits

If you are measuring a single variable y as a function of more than one variable — say, a *vector* of variables $\mathbf{x}$ — then your basis functions will be functions of a vector, $X_0(\mathbf{x}), \dots, X_{M-1}(\mathbf{x})$. The χ^2 merit function is now

$$\chi^2 = \sum_{i=0}^{N-1} \left[\frac{y_i - \sum_{k=0}^{M-1} a_k X_k(\mathbf{x}_i)}{\sigma_i} \right]^2 \quad (15.4.24)$$

All of the preceding discussion goes through unchanged, with x replaced by $\mathbf{x}$. In fact, we anticipated this in the coding of `Fitsvd`, above, which can do multidimensional general linear fits as easily as one-dimensional. Here is how:

A second, overloaded, constructor in `Fitsvd` substitutes a matrix `xx` for what was previously a vector. The rows of the matrix are the `ndat` data points. The number of columns is the dimensionality of the space (that is, of $\mathbf{x}$). Similarly, the user-supplied function `funks` now takes a vector argument, an $\mathbf{x}$. A simple example (fitting a quadratic function to data in two dimensions) might be

```

VecDoub quadratic2d(VecDoub_I &xx) {
    VecDoub ans(6);
    Doub x=xx[0], y=xx[1];
    ans[0] = 1;
    ans[1] = x; ans[2] = y;
    ans[3] = x*x; ans[4] = x*y; ans[5] = y*y;
    return ans;
}

```

Be sure that the argument of your user function has exactly the type “`VecDoub_I &`” (and not, for example, “`VecDoub &`” or “`VecDoub_I`”), since strict C++ compilers are picky about this.

The two constructors in `Fitsvd` communicate to `fit` whether data points are one-dimensional or multidimensional by setting either `xmd` or `x` to `NULL`. This explains the oddity that `x` was bound to the user data as a pointer, while `y` and `sig` were bound as references. (Yes, we know this is a bit of a hack!)

CITED REFERENCES AND FURTHER READING:

- Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), Chapter 7.
- Lupton, R. 1993, *Statistics in Theory and Practice* (Princeton, NJ: Princeton University Press), Chapter 11.
- Lawson, C.L., and Hanson, R. 1974, *Solving Least Squares Problems* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 1995 (Philadelphia: S.I.A.M.).
- Monahan, J.F. 2001, *Numerical Methods of Statistics* (Cambridge, UK: Cambridge University Press), Chapter 5.